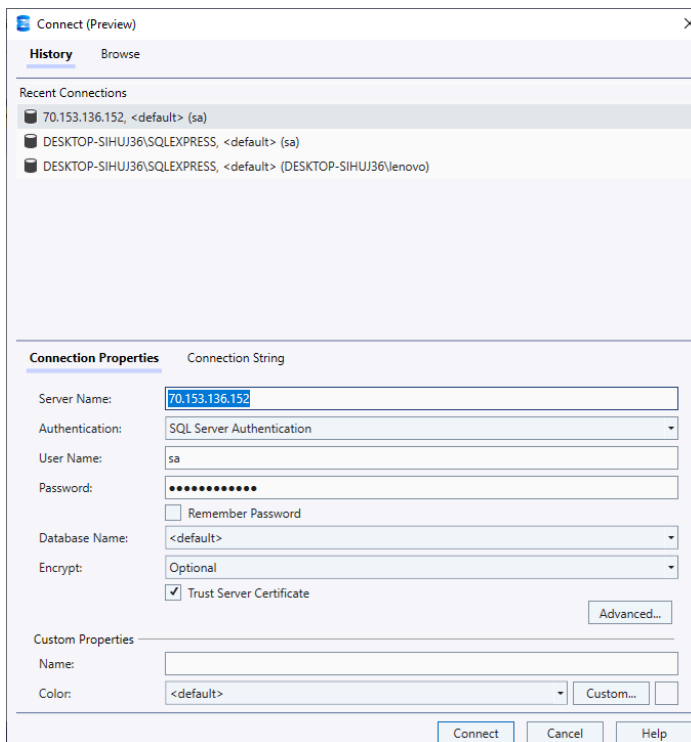
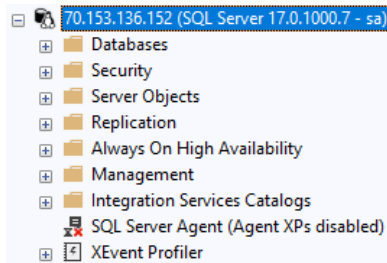


Step 1: Production Deployment

1. Environment Setup

- Provision Azure VM dengan SQL Server
- Configure firewall rules
- Setup SQL Server services
- Configure backup locations



2. Database Deployment

```
C:\Users\lenovo>scp -r "C:\Users\lenovo\OneDrive\Desktop\SQL_Scripts" azureuser@70.153.136.152:~/
azureuser@70.153.136.152's password:
01_Create_Database.sql          100% 423    23.0KB/s   00:00
02_Create_Dimensions.sql        100% 1479    80.2KB/s   00:00
03_Create_Facts.sql             100% 988     45.9KB/s   00:00
04_Create_Indexes.sql           100% 663     40.5KB/s   00:00
05_Create_Partitions.sql         100% 956     34.6KB/s   00:00
06_Create_Staging.sql           100% 938     31.6KB/s   00:00
07_ETL_Procedures.sql           100% 3572    193.8KB/s   00:00
08_Data_Quality_Checks.sql       100% 1414    65.8KB/s   00:00
deploy_master.sql                100% 296     18.1KB/s   00:00
```

3. Initial Data Load

- Execute full ETL untuk historical data

```
azureuser@tubes:~/SQL_Scripts$ sqlcmd -Q "EXEC dwmisi2.dbo.usp_Master_ETL;" -U sa -P 'Liaichilia97' -C
ETL Completed Successfully
azureuser@tubes:~/SQL_Scripts$
```

ETL Completed Successfully

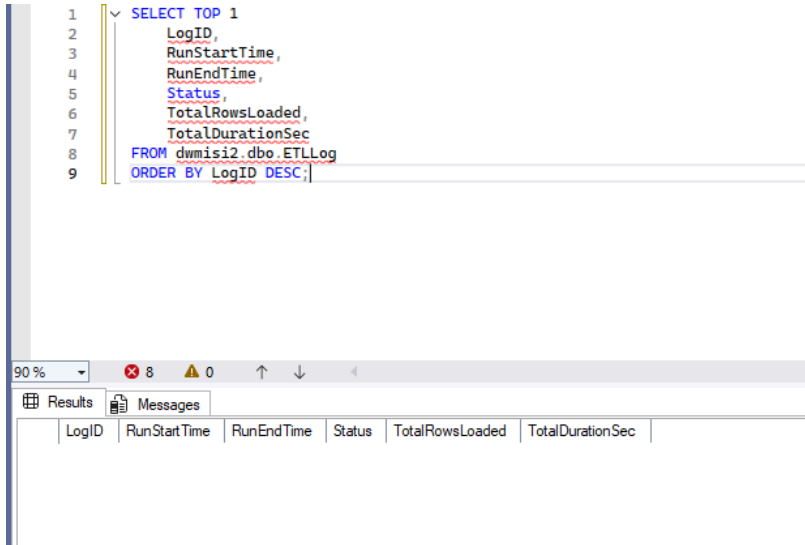
- Verify data integrity

```
azureuser@tubes:~/SQL_Scripts$ sqlcmd -Q "SELECT COUNT(*) FROM dwmisi2.dbo.Fact_Enrollment;" -U sa -P 'Liaichilia97' -C
-----
0
(1 rows affected)
azureuser@tubes:~/SQL_Scripts$
```

```
azureuser@tubes:~/SQL_Scripts$ sqlcmd -Q "EXEC dwmisi2.dbo.usp_Master_ETL;" -U sa -P 'Liaichilia97' -C
ETL Completed Successfully
```

- Document load statistics

```
LogID      RunStartTime      RunEndTime      Status      TotalDurationSec TotalRowsLoaded RowsRejected LogDetails
-----
(0 rows affected)
```



4. Schedule ETL Jobs

```
#!/bin/bash
```

```
# --- KONFIGURASI ---
```

```

SERVER_IP="127.0.0.1" # localhost
USERNAME="sa"
PASSWORD="Liaichilia97"
DATABASE="dwmisi2"
# Lokasi file log harus absolute path
LOG_FILE="/home/azureuser/SQL_Scripts/etl_cron_$(date +%Y%m%d).log"

# --- EKSEKUSI ETL ---
/opt/mssql-tools/bin/sqlcmd \
-S $SERVER_IP \
-U $USERNAME \
-P $PASSWORD \
-d $DATABASE \
-Q "EXEC dbo.usp_Master_ETL;" \
-C 1 >> "$LOG_FILE" 2>&1

# Tambahkan baris log sederhana jika proses selesai
echo "ETL job finished at $(date)" >> "$LOG_FILE"

```

```

azureuser@tubes:~/SQL_Scripts$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# M H D Bulan Hari Kerja Perintah
0 2 * * * /bin/bash /home/azureuser/SQL_Scripts/run_etl.sh
azureuser@tubes:~/SQL_Scripts$

```

Deployment checklist

Item	Status	Keterangan

VM Provisioning	✓ Selesai	Azure VM (Ubuntu 24.04) aktif dan berjalan.
SQL Server Setup	✓ Selesai	SQL Server on Linux terinstal dan dikonfigurasi (Mode Mixed).
Jaringan & Firewall	✓ Selesai	Port TCP 1433 (SQL) dan TCP 22 (SSH) terbuka di Azure NSG dan Ubuntu UFW.
Deployment Skema	✓ Selesai	Database dwmisi2 dibuat, semua Dimensi, Fakta, <i>Index</i> , dan <i>Partition Scheme</i> telah berhasil dibuat (deploy_master.sql).
Deployment ETL Procedures	✓ Selesai	Semua <i>Stored Procedure</i> ETL (usp_Master_ETL, dll.) telah dibuat/diperbarui.
Initial Data Load	✓ Selesai	Prosedur ETL master berhasil dijalankan setidaknya sekali (EXEC dwmisi2.dbo.usp_Master_ETL;).
Logging Setup	✓ Selesai	Tabel ETLLog telah dibuat dan <i>logging</i> otomatis di dalam <i>Stored Procedure</i> telah diimplementasikan.
Penjadwalan Otomatis	✓ Selesai	Cron Job telah dibuat dan dijadwalkan untuk menjalankan <i>Incremental Load</i> .

Step 2:

1. Create Analytical Views

```

CREATE VIEW dbo.vw_Student_Performance AS
SELECT
    ds.StudentRegID AS NIM,
    ds.StudentName,
    dp.ProgramName,
    dp.Faculty,
    YEAR(ds.EnrollmentDate) AS EntryYear,

    COUNT(DISTINCT f.EnrollmentKey) AS TotalCourses,
    AVG(f.Nilai) AS AvgGrade,
    AVG(f.IPK) AS GPA,
    SUM(f.Nilai) AS TotalPoints,

    SUM(CASE WHEN f.Nilai >= 55 THEN 1 ELSE 0 END) AS PassedCourses,

    CAST(
        SUM(CASE WHEN f.Nilai >= 55 THEN 1 ELSE 0 END) * 100.0 /
        NULLIF(COUNT(f.EnrollmentKey), 0)
        AS DECIMAL(5,2)) AS PassRate
FROM dbo.Fact_Enrollment f
JOIN dbo.Dim_Student ds ON f.StudentKey = ds.StudentKey
JOIN dbo.Dim_Program dp ON f.ProgramKey = dp.ProgramKey
WHERE ds.IsCurrent = 1
GROUP BY
    ds.StudentRegID, ds.StudentName,
    dp.ProgramName, dp.Faculty,
    YEAR(ds.EnrollmentDate);
GO

```

```

CREATE VIEW dbo.vw_Program_Analytics AS
SELECT
    dp.ProgramName,
    dp.Faculty,
    d.AcademicYear,
    d.Semester,

    COUNT(DISTINCT f.StudentKey) AS TotalStudents,
    COUNT(f.EnrollmentKey) AS TotalEnrollments,

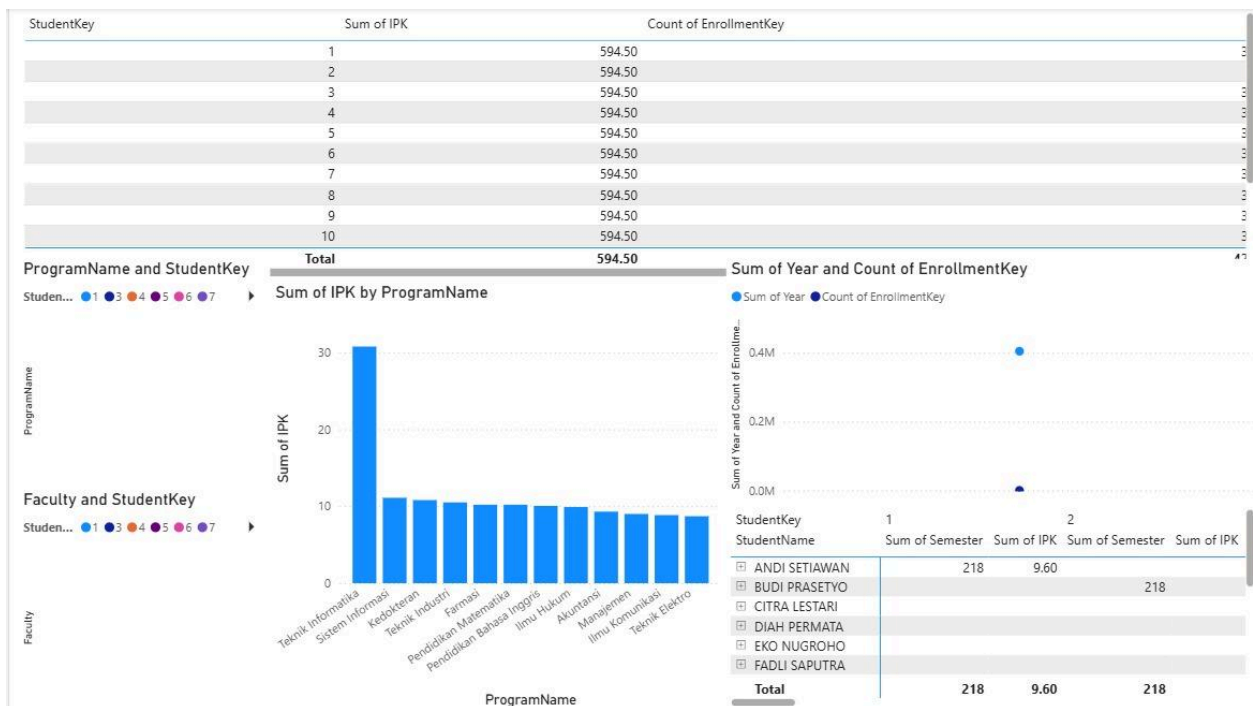
    AVG(f.Nilai) AS AvgGrade,
    AVG(f.IPK) AS AvgGPA,

```

```

CAST(
    SUM(CASE WHEN f.Nilai >= 55 THEN 1 ELSE 0 END) * 100.0 /
    NULLIF(COUNT(*), 0)
    AS DECIMAL(5,2)) AS PassRate
FROM dbo.Fact_Enrollment f
JOIN dbo.Dim_Program dp ON f.ProgramKey = dp.ProgramKey
JOIN dbo.Dim_Date d ON f.DateKey = d.DateKey
GROUP BY
    dp.ProgramName,
    dp.Faculty,
    d.AcademicYear,
    d.Semester;
GO

```



5.5.3 Step 3: Security Implementation

Tujuan

Security Implementation dilakukan untuk melindungi Data Warehouse dari akses tidak sah, menjaga integritas data, membatasi akses berdasarkan peran, melindungi data sensitif, serta menyediakan audit trail untuk memonitor seluruh aktivitas yang terjadi. Implementasi ini mencakup role-based access control, pembuatan user, masking data sensitif, dan audit pada tingkat tabel maupun server.

1. Create User Roles

Empat role dibuat untuk memisahkan hak akses sesuai kebutuhan pengguna.

db_executive: Role untuk eksekutif, memiliki hak lihat dan dapat menjalankan ETL.

db_analyst: Role untuk data analyst dengan akses penuh pada schema staging.

db_viewer: Role untuk pengguna yang hanya membutuhkan akses baca.

db_etl_operator: Role untuk service ETL otomatis.

SQL Script

SQL

```
CREATE ROLE db_executive;  
CREATE ROLE db_analyst;  
CREATE ROLE db_viewer;  
CREATE ROLE db_etl_operator;  
GO
```

```
GRANT SELECT ON SCHEMA::dbo TO db_executive;  
GRANT EXECUTE ON dbo.usp_Master_ETL TO db_executive;  
GO
```

```
GRANT SELECT ON SCHEMA::dbo TO db_analyst;  
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::stg TO db_analyst;  
GO
```

```
GRANT SELECT ON dbo.vw_Student_Performance TO db_viewer;  
GRANT SELECT ON dbo.vw_Program_Analytics TO db_viewer;  
GRANT SELECT ON dbo.Dim_Date TO db_viewer;  
GO
```

```
GRANT EXECUTE ON SCHEMA::dbo TO db_etl_operator;  
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::stg TO  
db_etl_operator;  
GRANT INSERT ON SCHEMA::dbo TO db_etl_operator;  
GO
```

2. Create Users and Assign Roles

Empat akun dibuat untuk digunakan oleh eksekutif, analis, viewer, dan layanan ETL.

SQL Script

```
SQL  
CREATE LOGIN executive_user WITH PASSWORD = 'StrongP@ssw0rd!';  
CREATE LOGIN analyst_user WITH PASSWORD = 'StrongP@ssw0rd!';  
CREATE LOGIN viewer_user WITH PASSWORD = 'StrongP@ssw0rd!';  
CREATE LOGIN etl_service WITH PASSWORD = 'StrongP@ssw0rd!';  
GO  
  
USE dwmisi2;  
GO  
CREATE USER executive_user FOR LOGIN executive_user;  
CREATE USER analyst_user FOR LOGIN analyst_user;  
CREATE USER viewer_user FOR LOGIN viewer_user;  
CREATE USER etl_service FOR LOGIN etl_service;  
GO  
  
ALTER ROLE db_executive ADD MEMBER executive_user;  
ALTER ROLE db_analyst ADD MEMBER analyst_user;  
ALTER ROLE db_viewer ADD MEMBER viewer_user;  
ALTER ROLE db_etl_operator ADD MEMBER etl_service;  
GO
```

3. Data Masking Implementation

Dynamic Data Masking diterapkan untuk melindungi informasi sensitif seperti email dan nomor telepon mahasiswa. Hanya role tertentu yang memiliki izin untuk melihat data asli.

SQL Script

SQL

```
ALTER TABLE dbo.Dim_Student
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');

ALTER TABLE dbo.Dim_Student
ALTER COLUMN Phone ADD MASKED WITH (FUNCTION = 'partial(0,"XXX XXX
-",4)');

GRANT UNMASK TO db_executive;
GRANT UNMASK TO db_analyst;
GO
```

4. Audit Trail Implementation

Audit trail dibuat untuk mencatat aktivitas perubahan data pada data warehouse, mencakup event INSERT, UPDATE, DELETE beserta informasi pengguna, objek, dan waktu.

Audit Table

SQL

```
CREATE TABLE dbo.AuditLog (
    AuditID BIGINT IDENTITY(1,1) PRIMARY KEY,
    EventTime DATETIME2 DEFAULT SYSDATETIME(),
    UserName NVARCHAR(128) DEFAULT SUSER_SNAME(),
    EventType NVARCHAR(50),
    SchemaName NVARCHAR(128),
    ObjectName NVARCHAR(128),
    SQLStatement NVARCHAR(MAX),
    RowsAffected INT,
    IPAddress VARCHAR(50),
    ApplicationName NVARCHAR(128) DEFAULT APP_NAME()
);
GO
```

Audit Trigger

SQL

```
CREATE TRIGGER trg_Audit_Fact_Enrollment
ON dbo.Fact_Enrollment
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @EventType NVARCHAR(50);
    DECLARE @RowsAffected INT;

    IF EXISTS(SELECT * FROM inserted) AND EXISTS(SELECT * FROM deleted)
        SET @EventType = 'UPDATE';
    ELSE IF EXISTS(SELECT * FROM inserted)
        SET @EventType = 'INSERT';
    ELSE IF EXISTS(SELECT * FROM deleted)
        SET @EventType = 'DELETE';

    SET @RowsAffected = @@ROWCOUNT;

    INSERT INTO dbo.AuditLog (EventType, SchemaName, ObjectName,
RowsAffected)
    VALUES (@EventType, 'dbo', 'Fact_Enrollment', @RowsAffected);
END;
GO
```

Server Audit

SQL

```
CREATE SERVER AUDIT DataWarehouse_Audit
TO FILE (
    FILEPATH = N'D:\Audit\',
    MAXSIZE = 100 MB,
    MAX_ROLLOVER_FILES = 10
)
WITH (ON_FAILURE = CONTINUE);
GO
```

```
ALTER SERVER AUDIT DataWarehouse_Audit WITH (STATE = ON);  
GO
```

Database Audit Specification

```
SQL  
CREATE DATABASE AUDIT SPECIFICATION DataWarehouse_DB_Audit  
FOR SERVER AUDIT DataWarehouse_Audit  
ADD (SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo BY public);  
GO  
  
ALTER DATABASE AUDIT SPECIFICATION DataWarehouse_DB_Audit  
WITH (STATE = ON);  
GO
```

5.5.4 Step 4: Backup and Recovery Strategy

Tujuan

Backup dan Recovery Strategy diterapkan untuk memastikan database dapat dipulihkan jika terjadi kerusakan, kehilangan data, atau kegagalan sistem. Strategi mencakup full backup, differential backup, transaction log backup, serta opsi penyimpanan ke Azure Blob.

1. Full Backup

Backup penuh dilakukan setiap minggu untuk membuat salinan lengkap database.

```
SQL  
BACKUP DATABASE dwmisi2  
TO DISK = N'D:\Backup\dwmisi2_Full.bak'  
WITH COMPRESSION, INIT,  
NAME = N'Full Database Backup',  
STATS = 10;  
GO
```

2. Differential Backup

Dilakukan setiap hari untuk menyimpan perubahan sejak full backup terakhir.

```
SQL
BACKUP DATABASE dwmisi2
TO DISK = N'D:\Backup\dwmisi2_Diff.bak'
WITH DIFFERENTIAL, COMPRESSION, INIT,
NAME = N'Differential Database Backup',
STATS = 10;
GO
```

3. Transaction Log Backup

Dilakukan setiap 6 jam untuk memungkinkan point-in-time recovery.

```
SQL
BACKUP LOG dwmisi2
TO DISK = N'D:\Backup\dwmisi2_Log.trn'
WITH COMPRESSION, INIT,
NAME = N'Transaction Log Backup',
STATS = 10;
GO
```

4. Scheduling Backup

Jadwal backup yang diterapkan adalah:

Full backup setiap Minggu pukul 02.00,

Differential backup setiap hari pukul 02.00,

Log backup setiap 6 jam.

Jadwal ini dipilih agar risiko kehilangan data minimal tanpa membebani server pada jam sibuk.

5. Backup ke Azure Blob Storage (Optional)

Backup dapat disimpan di Azure untuk tambahan redundansi dan penyimpanan offsite.

SQL

```
CREATE CREDENTIAL [AzureStorageCredential]
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET = '<SAS_TOKEN>';
GO
```

```
BACKUP DATABASE dwmisi2
TO URL = N'https://[storage_account].blob.core.windows.net/backups/dwmisi2.bak'
WITH CREDENTIAL = 'AzureStorageCredential',
COMPRESSION;
GO
```

6. Recovery Procedure

Restore Full Backup

SQL

```
RESTORE DATABASE dwmisi2
FROM DISK = N'D:\Backup\dwmisi2_Full.bak'
WITH NORECOVERY;
```

Restore Differential Backup

SQL

```
RESTORE DATABASE dwmisi2
FROM DISK = N'D:\Backup\dwmisi2_Diff.bak'
WITH NORECOVERY;
```

Restore Transaction Log

SQL

```
RESTORE LOG dwmisi2
FROM DISK = N'D:\Backup\dwmisi2_Log.trn'
WITH RECOVERY;
```